Guía de Clase: El Pseudo-Vector Dinámico y FILE en C

Objetivo pedagógico: Comprender el funcionamiento interno de las colecciones abstractas y asimilar el concepto de archivos en C a través de una estructura conocida.

Etapa 1: Funciones Sueltas y Arreglo Común

Manipulación de un arreglo tradicional utilizando pasaje de parámetros por referencia (punteros) para controlar el tamaño lógico.

```
#include <stdio.h>
#define MAX 100

void agregar(int vec[], int *tam, int valor) {
    if (*tam < MAX) {
        vec[*tam] = valor;
        (*tam)++;
    } else {
        printf("Error: Lista llena\n");
    }
}

void borrar(int vec[], int *tam, int posicion) {
    if (posicion >= 0 && posicion < *tam) {
        for (int i = posicion; i < *tam - 1; i++) {
            vec[i] = vec[i + 1];
        }
        (*tam)--;
    } else {
        printf("Error: Posicion invalida\n");
    }
}
```

Etapa 2: Estructura Integrada con Permisos

Encapsulamos los datos, la cantidad y los permisos de operación dentro de un mismo tipo de dato. El campo puede_borrar actúa como analogía temprana de los modos de apertura de archivos.

```
typedef struct {
    int datos[MAX];
    int cantidad;
    int puede_borrar;   // 1 = se puede borrar, 0 = no se puede
} ListaPython;

void agregarEnEstructura(ListaPython *lista, int valor) {
    if (lista->cantidad < MAX) {
        lista->datos[lista->cantidad] = valor;
        lista->cantidad++;
    }
}

void borrarEnEstructura(ListaPython *lista, int posicion) {
    if (lista->puede_borrar == 0) {
        printf("Error: No se permite borrar\n");
        return;
    }
    if (posicion >= 0 && posicion < lista->cantidad) {
        for (int i = posicion; i < lista->cantidad - 1; i++) {
            lista->datos[i] = lista->datos[i + 1];
        }
        lista->cantidad--;
    }
}
```

Ejemplos de instanciación de estructuras

Se crean varias estructuras con distintos permisos para mostrar cómo el modo de uso limita las operaciones posibles.

```
ListaPython lista1 = {{10, 20, 30}, 3, 1};           // puede borrar
ListaPython lista2 = {{5, 15}, 2, 0};                // no puede borrar
ListaPython lista3 = {{100, 200, 300, 400}, 4, 1};   // puede borrar

// Uso
agregarEnEstructura(&lista1, 40);   // permitido
borrarEnEstructura(&lista2, 0);     // muestra error
```

Etapa 3: Operaciones de Posición (Agregar, Buscar, Insertar)

Estas funciones controlan la posición lógica dentro de la estructura. Más adelante se relacionarán con fseek y ftell.

```
int buscar(ListaPython *lista, int valor) {
    for (int i = 0; i < lista->cantidad; i++) {
        if (lista->datos[i] == valor) return i;
    }
    return -1;
}

void insertar(ListaPython *lista, int posicion, int valor) {
    if (posicion >= 0 && posicion <= lista->cantidad && lista->cantidad < MAX) {
        for (int i = lista->cantidad; i > posicion; i--) {
            lista->datos[i] = lista->datos[i - 1];
        }
        lista->datos[posicion] = valor;
        lista->cantidad++;
    }
}
```

Etapa 4: Analogía con Archivos

Una vez comprendida la estructura, se establece la analogía:

- Crear una variable ListaPython ≈ abrir un archivo (fopen).
- El campo puede_borrar ≈ modo de apertura ("r", "w", "a").
- buscar / insertar ≈ mover la posición con fseek.
- Liberar o terminar el uso ≈ fclose.

Etapa 5: Persistencia

Se guarda y recupera la estructura en un archivo. Aquí se introducen formalmente fopen, fclose y los modos de apertura.

```
void guardar(ListaPython *lista, char *nombre) {
    FILE *archivo = fopen(nombre, "w");
    if (archivo != NULL) {
        fprintf(archivo, "%d\n", lista->cantidad);
        for (int i = 0; i < lista->cantidad; i++) {
            fprintf(archivo, "%d ", lista->datos[i]);
        }
        fclose(archivo);
    }
}

void cargar(ListaPython *lista, char *nombre) {
    FILE *archivo = fopen(nombre, "r");
    if (archivo != NULL) {
        fscanf(archivo, "%d", &lista->cantidad);
        for (int i = 0; i < lista->cantidad; i++) {
```

```
            fscanf(archivo, "%d", &lista->datos[i]);
        }
        fclose(archivo);
    }
}
```

Orden didáctico final

1. Funciones sueltas + arreglo
2. Estructura con permisos (puede_borrar)
3. Operaciones de posición (buscar, insertar)
4. Analogía estructura ↔ archivo
5. Persistencia y modos de apertura (fopen / fclose)